



**DEPARTAMENTO
DE COMPUTACION**

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo Práctico

Diseño de un Módulo de Shelf Selection para un Sistema Robótico G2P

1 de septiembre de 2026

Optimización aplicada a la distribución logística

Integrante	LU	Correo electrónico
Aguirre, Victoria	626/19	viaguirre1001@gmail.com
Enrich, Mauricio	458/22	mauricioenrich12@gmail.com
Pasquet, Felipe	1084/22	felipe.pasquet@gmail.com
Maurer, Milagros	42/22	milagros.maurer@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

<http://www.exactas.uba.ar>

1. Fase 1: Análisis del Problema y Requerimientos

El objetivo de este trabajo es enfocarnos en el componente WES (Warehouse Execution System), que traduce las directivas del WMS en tareas ejecutables por la capa inferior. Nuestro principal problema a tratar va a ser el módulo Shelf Selection (SS), que decide qué estanterías deben ser transportadas a las estaciones de picking para procesar las órdenes pendientes. El enfoque para plantear este problema y plantear la función objetivo es el siguiente:

- Maximizar Picks Per Face: Queremos que haya densidad de items por cada cara del rack que elegimos para transportar. Si traducimos esto, sería que haya una gran cantidad de órdenes que se satisfagan con la menor cantidad de caras del rack posibles. Como cada ola tiene que cumplir con la entrega de N órdenes, pensamos que lo correcto fuera $\max \frac{N}{\sum_i \sum_j Q_{ij}}$, pero como N es una constante, es equivalente a $\min \sum_i \sum_j Q_{ij}$, lo cuál sería, minimizar las caras de rack usadas por ola.
- Cumplimiento de SLA: Debido a que las órdenes tienen vencimiento, es importante que para cada pedido ese plazo no se venza. Es por eso que en la función objetivo, planteamos una variable D_k que nos diga si la orden k es entregada en plazo post vencimiento. Así, es como queremos minimizar la cantidad de órdenes entregadas fuera del plazo correspondiente y poner una penalización P por cada entrega que no cumpla con su plazo.
- Racks: La idea es tratar de minimizar la cantidad de racks que se movilizan/utilizan. Los racks fríos tienen la mayor penalización. El costo de traer un rack frío es 5 y uno tibio es 3. Si usamos otras caras de un rack que ya trajimos, el costo de las siguientes caras es 1 (se lo denomina rack caliente).

Variables binarias:

$$P_{kij} = 1 \quad \text{si la orden } k \text{ se satisface por el rack } i \text{ cara } j \quad (1)$$

$$Q_{ij} = 1 \quad \text{si se usa rack } i \text{ cara } j \quad (2)$$

$$F_i = 1 \quad \text{si el rack } i \text{ es frío y es utilizado} \quad (3)$$

$$T_i = 1 \quad \text{si el rack } i \text{ es tibio y es utilizado} \quad (4)$$

$$D_k = 1 \quad \text{si la orden } k \text{ se venció} \quad (5)$$

$$P_{kij}, Q_{ij}, F_i, T_i, D_k \in \{0, 1\}$$

Función objetivo:

$$\text{Min} : \sum_i \sum_j Q_{ij} + 5 \sum_i F_i + 3 \sum_i T_i + \sum_i (\sum_j Q_{ij} - (F_i + T_i)) + P \sum_k D_k$$

Restricciones:

$$\sum_i \sum_j P_{kij} \leq 1 \quad \forall k$$

$$\sum_k \sum_i \sum_j P_{kij} = N$$

$$\begin{aligned}\sum_k P_{kij} &\leq N Q_{ij} \quad \forall i, j \\ \sum_j Q_{ij} &\leq 4F_i + 4T_i \quad \forall i \\ T_i + F_i &\leq 1 \quad \forall i \\ \sum_i T_i &\leq 0,2R\end{aligned}$$

Siendo R la cantidad total de racks en el depósito, P una constante de penalización y N la cantidad de órdenes devueltas en el ciclo.

Explicación de las restricciones:

- $\sum_i \sum_j P_{kij} \leq 1 \quad \forall k$: cada orden k sólo puede satisfacerse por una única cara de un único rack.
- $\sum_k \sum_i \sum_j P_{kij} = N$: el módulo SS debe devolver exactamente N órdenes en cada ciclo
- $\sum_k P_{kij} \leq N Q_{ij} \quad \forall i, j$: vincula las variables P_{kij} con Q_{ij} , activando una cara ($Q_{ij} = 1$) si al menos una orden se satisface en ella.
- $\sum_j Q_{ij} \leq 4F_i + 4T_i \quad \forall i$: garantiza que solo puedan activarse caras de racks fríos o tibios y que por lo menos uno de ellos esté activo.
- $T_i + F_i \leq 1 \quad \forall i$: un mismo rack no puede ser simultáneamente tibio y frío en el mismo ciclo.
- $\sum_i T_i \leq 0,2R$: como máximo el 20 % de los racks totales pueden ser tibios.

2. Fase 2: Exploración de Datos y Enfoques de Solución

Algunos números relevantes:

- La cantidad de items en stock es 421.357 items.
- La cantidad de items pedidos es 390.855 items.
- Hay en total 2.087 racks.
- Hay en total 8.311 caras.

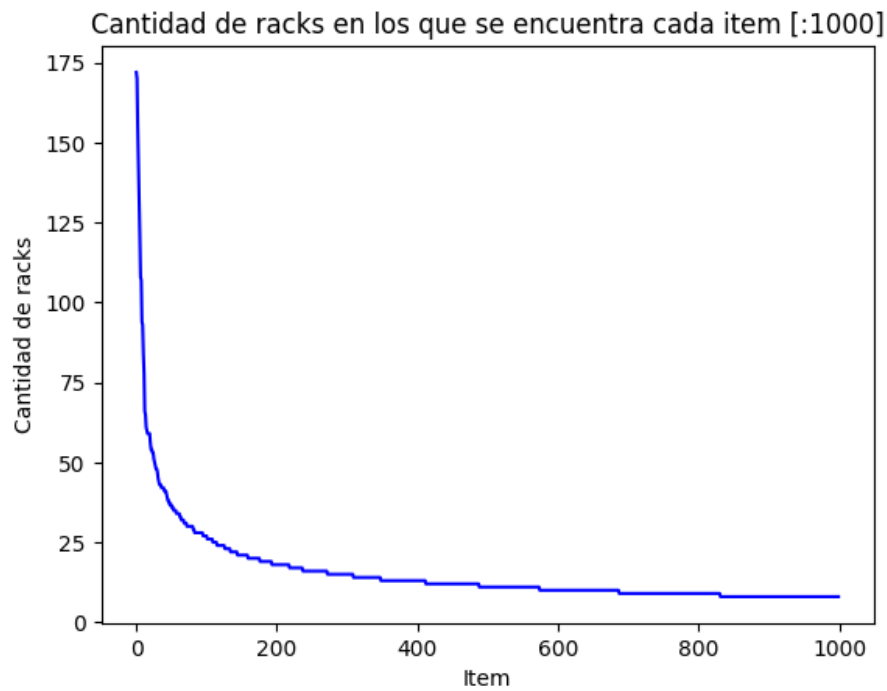


Figura 1: Para cada ítem, se calcula la cantidad de racks en los que se encuentra. Se toman los primeros 1.000 ítems que tienen mayor cantidad de apariciones en los racks.

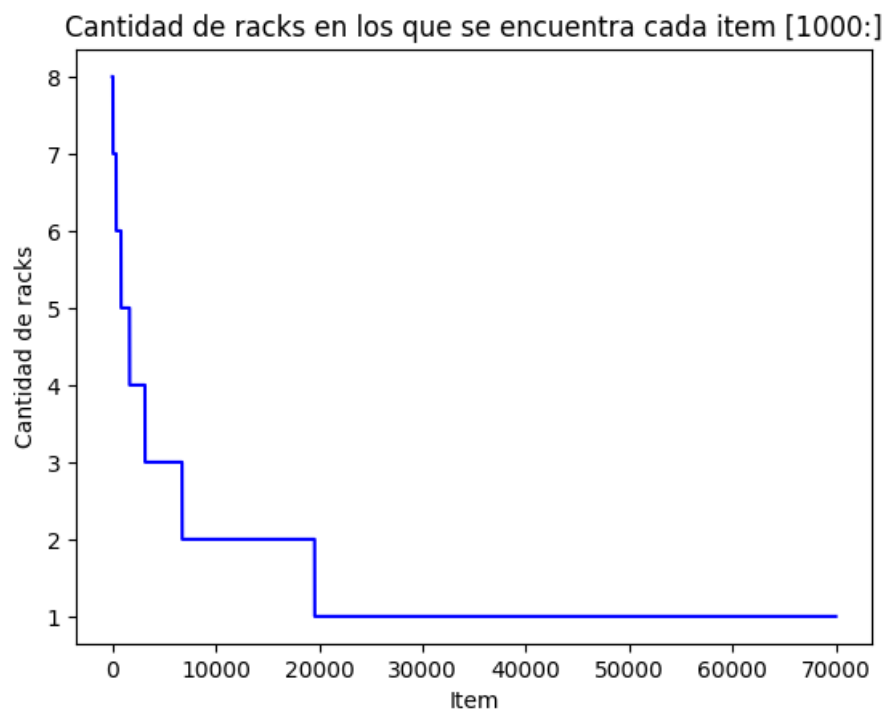


Figura 2: Para cada ítem, se calcula la cantidad de racks en los que se encuentra. Se toman los últimos 69.000 ítems que tienen mayor cantidad de apariciones en los racks.

Se puede notar en la Figura 2 como hay aproximadamente 50.000 ítems que solo se encuentran en un rack. También se puede ver que hay algunos ítems que se encuentran más dispersos

en cuanto a la cantidad de racks en los que se encuentran. Por ejemplo, en la Figura 1 se puede observar como hay un ítem que se encuentra en 175 racks distintos.

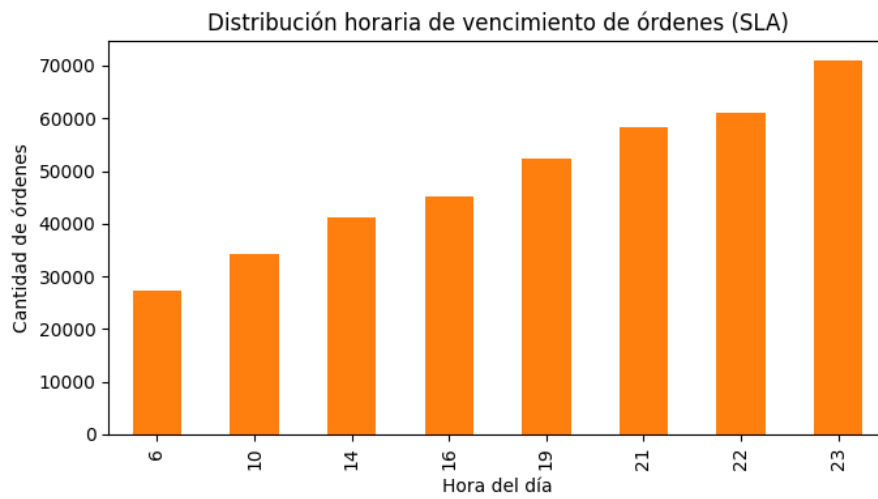


Figura 3: Para cada orden del backlog vemos su horario de vencimiento.

La Figura 3 muestra que los vencimientos de las órdenes no son uniformes, concentrándose en determinados horarios. Esto da una idea de cuáles son las órdenes que se deben priorizar.

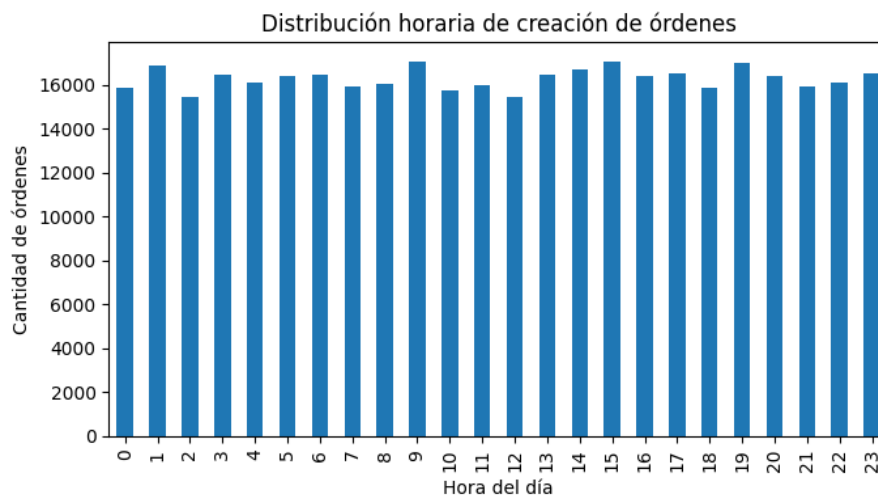


Figura 4: Para cada orden del backlog vemos su horario de creación.

En cambio, la creación de órdenes presenta una distribución más uniforme, lo que implica un flujo constante de nuevos pedidos.

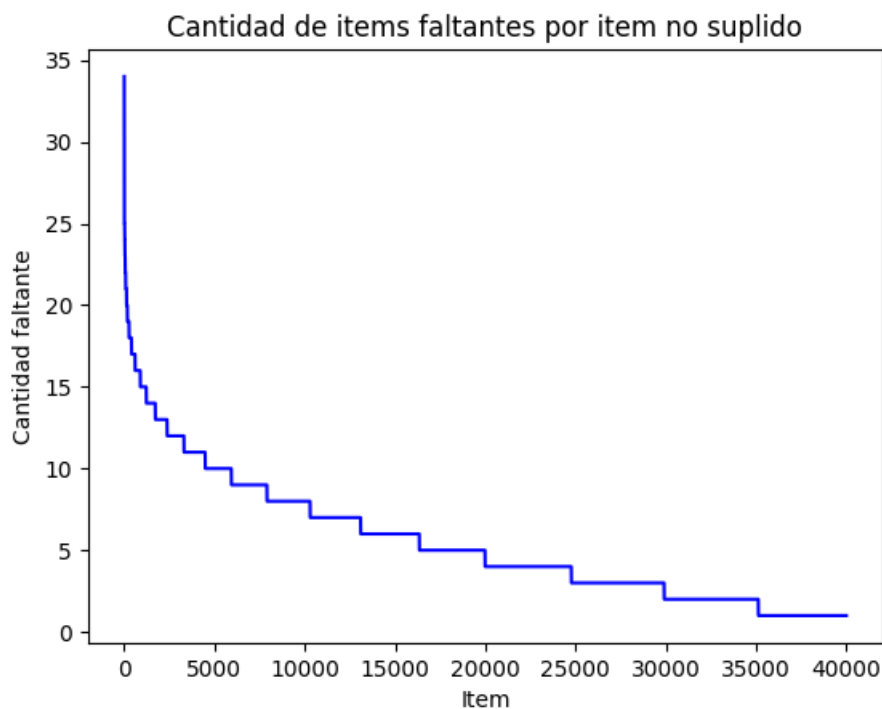


Figura 5: Productos que tienen faltante de stock

Observamos que de los 390.855 pedidos, 219.109 pedidos no podrán ser suplidos pues no hay stock suficiente. Dado que los pedidos para los cual no hay stock no vamos a poder cumplirlos y no hay nada que podamos hacer al respecto, no los vamos a tomar en cuenta para el resto del trabajo. A medida que pasa el tiempo y se agota el stock, cada vez vamos a procesar menos pedidos. Podemos ver en el siguiente gráfico la creación de órdenes luego de hacer esta limpieza:

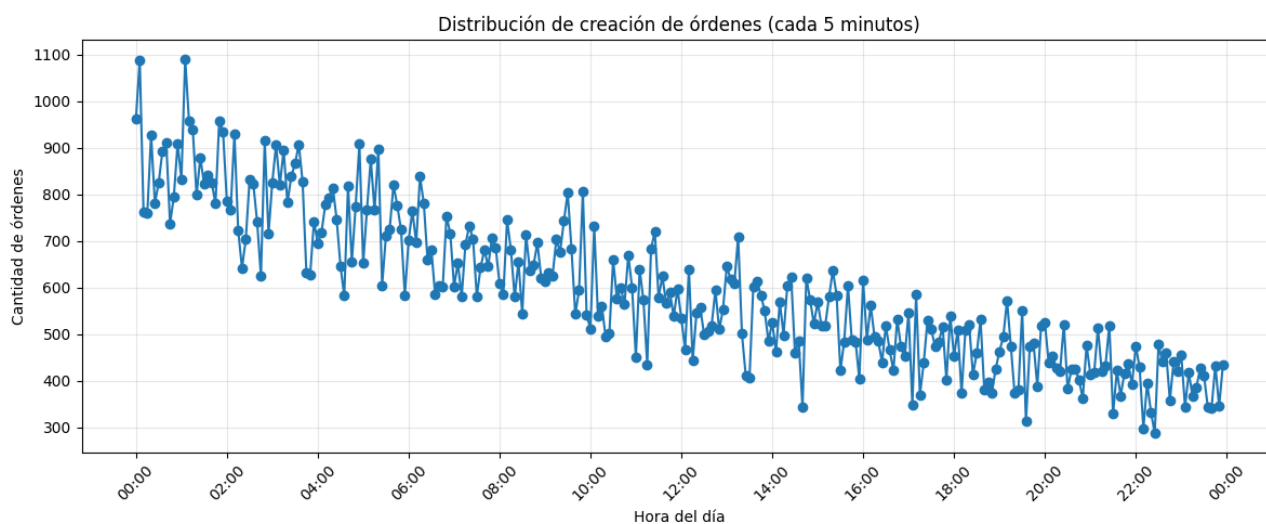


Figura 6: Creación de órdenes a medida que se agota el stock

En esta figura vemos la cantidad de órdenes creadas cada 5 minutos. Efectivamente se crean menos órdenes a medida que pasa el tiempo y la razón de esto, como recién explicamos, es

que a medida que pasa el tiempo y se agota el stock, no tomamos en cuenta las órdenes que no vamos a poder cumplir. Algo importante que observamos en este gráfico es que casi no hay intervalos de 5 minutos en los que se crean más de 1000 órdenes. Esto es importante pues el N (cantidad de órdenes que tenemos que cumplir en un ciclo) es un número random entre 1000 y 2000, sin embargo si no hay suficientes órdenes creadas, como en la mayoría de los casos, no vamos a poder cumplir esta condición.

Del análisis anterior se concluye que el stock presenta alta variabilidad: muchos ítems aparecen en un solo rack, mientras que otros aparecen en más de 100 racks diferentes. Esto limita la posibilidad de agrupar todos los pedidos en pocas caras, pero permite aprovechar los racks con mayor abundancia de un ítem para maximizar *Picks per Face*.

A su vez, la concentración de vencimientos en ciertos horarios obliga a priorizar aquellas órdenes con `due_date` más cercano.

Debido al volumen de datos (390.855 órdenes y 2.087 racks), un enfoque con programación entera sería inviable de resolver cada ciclo en menos de 5 minutos. Lo planteado en la Fase 1 implicaría resolver un modelo con millones de variables binarias (Por ej: 6.779.402.253.735 variables P_{kij}). Por lo tanto, una heurística constructiva es el enfoque más viable. Dentro de ella, se pueden aplicar las siguientes reglas:

- Priorizar órdenes pendientes y próximas a vencer
- Seleccionar racks con mayor cobertura de ítems
- Agrupar pedidos del mismo ítem
- Aprovechar racks calientes o tibios antes de traer racks fríos.

3. Fase 3: Propuesta de Arquitectura.

El módulo Shelf Selection (SS) se encuentra dentro de la capa WES, que actúa como intermediaria entre el WMS (que gestiona el inventario y las órdenes) y el Task Manager (TM), encargado de generar las tareas operativas para el Robot Control System (RCS).

En cada ciclo de ejecución (cada 5 minutos), el SS recibe del WMS una foto del stock (`stock.json`) y el backlog de órdenes (`backlog.json`), junto con las tareas pendientes del ciclo anterior (desde el TM) y el parámetro N que indica cuántas órdenes deben procesarse. Con esta información, el algoritmo del SS prioriza las órdenes más urgentes (según su `due_date`), selecciona racks minimizando los costos de movimiento (frío, tibio o caliente), y que reutiliza racks ya movilizados para procesar múltiples órdenes en la misma cara, maximizando *Picks per Face*.

El resultado del SS es un *pool de tareas* agrupado por rack y cara, junto con el costo total y la actualización de los racks tibios. Esta información es enviada al TM que se encarga de convertir las órdenes seleccionadas en un conjunto de tareas ordenadas, con origen y destino para ser entregadas al RCS para su ejecución. Cuando termina el ciclo, las tareas no completadas son devueltas al SS como **pendientes**, y el backlog actualizado (sin las órdenes ya asignadas) se mantiene en el SS para el siguiente ciclo.

4. Fase 4: Propuesta de un algoritmo para el SS.

Como se dijo en varias ocasiones a lo largo del informe, es muy costoso computacionalmente resolver este problema con programación lineal entera. Es por eso que se decidió crear una heurística con una estrategia Greedy, para que sea factible ejecutarlo en el ciclo de 5 minutos del Task Manager.

Como primer paso, se hace una limpieza del backlog. A lo largo del proyecto se pudo notar que, muchas de las órdenes del backlog no podían ser suplidas por falta de stock. Es por eso que se tomó la decisión de borrar aquellas que no se pueden cumplir y mantener en el backlog las órdenes que sí se pueden cumplir por stock.

La estrategia principal del algoritmo se basa en la clasificación de racks según su estado (fríos, tibios o calientes), que determina el costo de movimentación de ellos. Se mantienen dos estructuras: una matriz `racks_calientes` que registra qué caras de qué racks fueron utilizadas en el ciclo actual ¹, y un array `racks_tibios` que marca los racks usados recientemente y mantenidos en el área de almacenamiento rápido. Las caras de racks calientes que ya se usaron, tienen costo 0 si se vuelven a usar; usar una nueva cara de un rack caliente tienen costo 1 y usar una cara de un rack tibio tiene costo 3. Los racks que no están ni calientes ni tibios se consideran fríos y cuestan 5. Para cada orden, la función `definir_rack` evalúa todas las ubicaciones posibles del item solicitado y elige la de menor costo, actualizando inmediatamente el estado de racks calientes para que órdenes posteriores en el mismo ciclo puedan aprovechar racks ya en uso y maximizar implícitamente el PPF.

Este algoritmo va a priorizar elegir racks tibios al principio del ciclo, y a medida que hay más racks calientes va a priorizar estos y en particular las *caras calientes*. El sesgo de nuestra heurística sería que asumimos que a medida que pasa el tiempo, los "mejores racks" van a ir quedando en la lista de racks tibios. Para que esto se cumpla necesitamos que la cantidad de ciclos sea grande. Por otro lado, asumimos que a medida que pasa el tiempo se van a utilizar las caras óptimas en cada ciclo (ie. las caras con más densidad de productos). Esto podría suceder si N, la cantidad de pedidos en un ciclo, es grande.

El procesamiento se realiza en dos fases: primero se asignan las **órdenes pendientes** del ciclo anterior (que no pudieron ser procesadas por el TM), y luego se completan las N órdenes requeridas seleccionando del backlog ordenado por `due_date` para priorizar las más urgentes. Si una orden se procesa después de su fecha de vencimiento, se agrega una penalización de costo 100. Al finalizar cada ciclo, los racks utilizados se marcan como tibios y en caso de que se supere el 20 % de almacenamiento se quitan los racks tibios menos recientes.

A continuación se presenta el pseudocódigo del algoritmo, incluidas sus funciones auxiliares.

¹`racks_calientes` es una matriz inicializada en ceros en cada ciclo. `racks_calientes[i][j] = 1` si ya se usó la cara j del rack i, 0 en caso contrario)

Algorithm 1 Procedimiento CONSTRUIR_UBICACION(*stock*)

```
1: Variables globales: ubicacion_items, totales_racks
2: ubicacion_items  $\leftarrow \{\}$  ▷ Diccionario vacío de listas
3: totales_racks  $\leftarrow \{\}$  ▷ Diccionario vacío de enteros
4: for all rack en stock do
5:   for all cara en stock[rack] do
6:     for all entrada en stock[rack][cara] do
7:       item  $\leftarrow$  entrada["Inventory ID"]
8:       cantidad  $\leftarrow$  int(entrada["Cantidad"])
9:       if cantidad  $\leq 0$  then
10:        continue
11:      end if
12:      Agregar {"rack" : rack, "cara" : cara, "cantidad" : cantidad} a
        ubicacion_items[item]
13:      totales_racks[rack]  $\leftarrow$  totales_racks[rack] + cantidad
14:    end for
15:  end for
16: end for
```

Algorithm 2 Función DEFINIR_RACK(*orden*, *racks_calientes*, *racks_tibios*)

```
1: Variables globales: ubicacion_items, totales_racks
2: item  $\leftarrow$  orden["item_id"]
3: order_id  $\leftarrow$  orden["order_id"]
4: if ubicacion_items[item] está vacío o stock total de item = 0 then
5:   return (None, 0)
6: end if
7: mejor_clasificacion  $\leftarrow$  50
8: index  $\leftarrow$  -1
9: for i  $\leftarrow$  0 hasta len(ubicacion_items[item]) - 1 do
10:   Obtener rack, cara, cantidad
11:   if cantidad = 0 then continue
12:   end if
13:   if racks_calientes[rack - 1][cara - 1] = 1 then
14:     clasificacion  $\leftarrow$  0
15:   else if  $\sum$  racks_calientes[rack - 1]  $\geq$  1 then
16:     clasificacion  $\leftarrow$  1
17:   else if racks_tibios[rack - 1] = 1 then
18:     clasificacion  $\leftarrow$  3
19:   else
20:     clasificacion  $\leftarrow$  5
21:   end if
22:   if clasificacion < mejor_clasificacion then
23:     mejor_clasificacion  $\leftarrow$  clasificacion
24:     index  $\leftarrow$  i
25:   end if
26: end for
27: if index = -1 then return (None, 0)
28: end if
29: Actualizar cantidad en ubicacion_items[item][index]
30: Actualizar totales_racks[rack]
31: racks_calientes[r - 1][c - 1]  $\leftarrow$  1
    mejor_clasificacion 0 tipo  $\leftarrow$  Rack Caliente"; costo  $\leftarrow$  0 1 tipo  $\leftarrow$  Rack Caliente";
    costo  $\leftarrow$  1 3 tipo  $\leftarrow$  Rack Tibio"; costo  $\leftarrow$  3 5 tipo  $\leftarrow$  Rack Frío"; costo  $\leftarrow$  5
32: tarea  $\leftarrow$  (orden, item, rack, cara, tipo)
33: return (tarea, costo)
```

Algorithm 3 Función PROCESAR_PENDIENTES(*pendientes*, *racks_calientes*, *racks_tibios*)

```

1: tareas  $\leftarrow []$ , costo_total  $\leftarrow 0$ 
2: for all orden en pendientes do
3:   (tarea, costo)  $\leftarrow$  DEFINIR_RACK(orden, racks_calientes, racks_tibios)
4:   if tarea  $\neq$  None then
5:     Agregar tarea a tareas
6:     costo_total  $\leftarrow$  costo_total + costo
7:   end if
8: end for
9: return (tareas, costo_total)

```

Algorithm 4 Función PROCESAR_BACKLOG(*racks_calientes*, *racks_tibios*, *faltantes*, *hora*)

```

1: tareas  $\leftarrow []$ , costo_total  $\leftarrow 0$ , penalizacion  $\leftarrow 0$ 
2: if faltantes  $\leq 0$  then return (tareas, 0)
3: end if
4: ordenes  $\leftarrow$  backlog ordenadas por due_date
5: ordenes_urgentes  $\leftarrow$  primeras faltantes
6: for all orden en ordenes_urgentes do
7:   (tarea, costo)  $\leftarrow$  DEFINIR_RACK(orden, racks_calientes, racks_tibios)
8:   if tarea existe then
9:     Agregar tarea, sumar costo, marcar orden completada
10:  end if
11: end for
12: while len(tareas) < faltantes y backlog no vacío do
13:   for all orden en copia del backlog do
14:     (tarea, costo)  $\leftarrow$  DEFINIR_RACK(orden, racks_calientes, racks_tibios)
15:     if tarea existe then
16:       Agregar tarea, costo, eliminar del backlog
17:       if hora > due_date then penalizacion  $\leftarrow$  penalizacion + 100
18:     end if
19:   end if
20: end for
21: end while
22: costo_total  $\leftarrow$  costo_total + penalizacion
23: return (tareas, costo_total)

```

Algorithm 5 Procedimiento ACTUALIZAR_TIBIOS(*tareas*, *racks_tibios*, *max_tibios*)

```
1: for all tarea en tareas do
2:    $r \leftarrow$  número de rack en tarea
3:    $racks\_tibios[r - 1] \leftarrow 1$ 
4: end for
5: if  $\sum(racks\_tibios) > max\_tibios$  then
6:    $exceso \leftarrow \sum(racks\_tibios) - max\_tibios$ 
7:   for  $i \leftarrow 0$  hasta  $len(racks\_tibios) - 1$  do
8:     if  $racks\_tibios[i] = 1$  then
9:        $racks\_tibios[i] \leftarrow 0$ 
10:       $exceso \leftarrow exceso - 1$ 
11:      if  $exceso = 0$  then break
12:    end if
13:  end if
14: end for
15: end if
```

Algorithm 6 Función principal SS(*stock*, *pendientes*, *N*, *racks_tibios*, *cant_racks*, *hora*)

```
1:  $racks\_calientes \leftarrow$  matriz [ $cant\_racks \times 4$ ] de ceros
2:  $max\_tibios \leftarrow int(0, 2 \times cant\_racks)$ 
3: (tareas_pend, costo_pend)  $\leftarrow PROCESAR\_PENDIENTES(pendientes, racks\_calientes, racks\_tibios)$ 
4: faltantes  $\leftarrow \max(0, N - len(tareas\_pend))$ 
5: (tareas_urg, costo_urg)  $\leftarrow PROCESAR\_BACKLOG(racks\_calientes, racks\_tibios, faltantes, hora)$ 
6: tareas  $\leftarrow tareas\_pend + tareas\_urg$ 
7: costo_total  $\leftarrow costo\_pend + costo\_urg$ 
8: ACTUALIZAR_TIBIOS(tareas, racks_tibios, max_tibios)
9: resultado  $\leftarrow \{\}$ 
10: for tarea in tareas do
11:    $resultados[(tarea[2], tarea[3])] \leftarrow tarea[0]['order\_id']$ 
12: end for
13: return (resultado, costo_total)
```

5. Fase 5: Métricas de evaluación de performance.

Se utilizaron las siguientes métricas para analizar la calidad de nuestras soluciones en cada ronda:

1. **Caras utilizadas en un ciclo** = Cantidad de caras usadas

Resultado obtenido: El promedio de caras utilizadas en un ciclo fue de **317.34**.

2. **Caras promedio por rack usado** = $\frac{\text{Cantidad de caras usadas}}{\text{Cantidad de racks usados}}$

Resultado obtenido: El promedio de caras utilizadas sobre racks utilizados fue de 1.09.

$$3. \text{ Picks Per Face} = \frac{\sum_i \sum_j Q_{ij}}{N} = \frac{\text{Cantidad de órdenes procesadas}}{\text{Cantidad de caras utilizadas}}$$

Resultado obtenido: PPF promedio de 1.89 a lo largo de las 24hs simuladas.

$$4. \text{ Costo promedio por orden} = \frac{\text{Costo total}}{\text{Órdenes cumplidas}}$$

Resultado obtenido: 2.25 fue el costo promedio en la simulación.

$$5. \text{ Tasa de cumplimiento de SLA} = \frac{\text{Órdenes procesadas a tiempo}}{\text{Total de órdenes procesadas}} \times 100 \%$$

Resultado obtenido: 100 % de las órdenes cumplieron con su SLA.

$$6. \text{ Tiempo de ejecución promedio de un ciclo} = \frac{\text{Tiempo de ejecución total}}{\text{Total de órdenes procesadas}}$$

Resultado obtenido: 0.35 segundos/ciclo.

Los resultados de las métricas evaluadas muestran un buen aprovechamiento de las caras y racks disponibles sin comprometer los tiempos. Además, el tiempo de ejecución promedio por ciclo confirma que la heurística puede ejecutarse en tiempo real dentro del intervalo de 5 minutos.

6. Conclusión

En este trabajo pudimos analizar los datos del stock y de los pedidos de un warehouse. Vimos que la mayoría de pedidos no podían cumplirse por faltante de stock y decidimos solo tomar los pedidos para los cuales había stock suficiente al momento de la creación del pedido, y desechar el resto. Esto llevó a que, a medida que pasa el tiempo y se agota el stock, tomemos y procesemos menos pedidos en cada ciclo de 5 minutos.

Creamos un algoritmo para el **SS**, que tiene un tiempo de ejecución muy bajo y permite ejecutarlo en tiempo real antes de empezar cada ciclo y logramos recopilar suficientes métricas para poder analizar la eficiencia del algoritmo.

En comparación con un enfoque de Programación Lineal Entera, que sería completamente inviable de resolver en ciclos de 5 minutos debido al número de variables binarias, la heurística propuesta ofrece un equilibrio adecuado entre calidad de solución y tiempo de ejecución.

Podríamos mejorar cómo elegimos las primeras caras de cada ciclo, porque actualmente el algoritmo no tiene ningún parámetro para elegir las primeras caras de cada ciclo y simplemente elegiría la primer cara que tenga el producto, eligiendo un rack tibio si fuera posible o un rack frío caso contrario. Estas primeras caras/racks probablemente sean subóptimas y pueden perjudicar mucho a la métrica de picks per face.